PRODUCT PLAN · ALL FIVE PHASES MEASURED

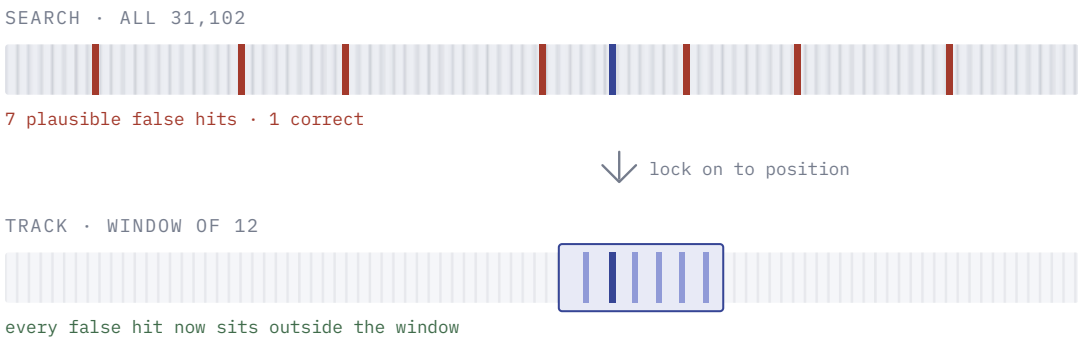
Scripture Live: *search, then follow*

Turning a single-file demo into a two-corpus product — Bible and Quran — that tracks a live reader hands-free, with no operator clicking ahead to guess where they are going.

PHASES 0–4	TARGET CORPORA	STACK	BROWSERS
complete · all gates met	31,102 + 6,236	Vite · TS · transformers.js	all four engines
BUILT FOR	**ON SCREEN**		
the projection operator	99.4% over 17 min		

The reframe that drives everything

> The demo *searches*. The product *searches once, then tracks*.
>
> Scanning the whole corpus on every utterance works at 155 verses because almost any match lands somewhere plausible. At 31,102 it collapses — `and it came to pass` appears roughly 450 times in the KJV, so a pure overlap scorer picks noise with confidence.
>
> Acquire position once, then lock and follow forward. If the reader is at Psalm 23:3, the next utterance is almost certainly 23:4 — the search space drops from 31,102 candidates to about twelve. **Context carries the weight the audio cannot**, which is what makes imperfect speech recognition survivable. Build this early; most of the architecture follows from it.
>
> There is also a hard ceiling that makes tracking mandatory rather than merely better. **4.5% of ayat and 1.5% of KJV verses are byte-identical to at least one other verse** — Ar-Rahman's refrain occurs 31 times, and “And the LORD spake unto Moses, saying” occurs 72 times. Text alone cannot separate these at any recognition quality; only knowing where the reader already was can.



Tracking is a small forced-alignment problem, not a search problem. Keep a periodic global check running underneath to catch genuine jumps.

Stack and browser support

Cross-browser is a consequence of owning the recognition

The Web Speech API is the only thing making the current demo Chrome-only. This is not a polish problem to be solved by feature detection, because two of the four engines have no implementation to detect.

	webkitSpeechRecognition	getUserMedia	WEBASSEMBLY
Chrome / Edge	yes — audio streamed to Google or Azure	yes	yes
Safari desktop	partial, unreliable for continuous use	yes	yes
iOS Safari	partial, heavily restricted	yes	yes
Firefox	never shipped	yes	yes

Firefox exposes a `media.webspeech.recognition.enable` preference with no backend behind it, and that is not changing. The other two columns are universal. **Capture the audio directly and run recognition in the page, and the browser question disappears.** Cross-browser support is not a later feature — it falls out of owning the recognition step.

That step has to be owned regardless: Arabic recitation will never work on a conversational model trained for Modern Standard Arabic, so Phase 3 forces it. Doing it in Phase 1 avoids solving browser support twice.

transformers.js is the specific route — Whisper through ONNX Runtime Web, using WebGPU where available (Chrome, Edge, Safari 18+, with Firefox rolling out) and falling back to WASM everywhere else. Quantized `whisper-tiny` is roughly 40 MB and `whisper-base` roughly 80 MB, cached in IndexedDB after first load. That buys every browser, offline operation, no per-minute cost, and no audio leaving the building — which matters when the audio is a congregation.

Keep the existing Web Speech path as an opportunistic fast lane on Chrome. It is already written and costs nothing. It must simply stop being the only path. iOS is the weakest target: HTTPS and a user gesture are required, background audio is restricted, and WASM is slow on older iPhones. It works, but budget real time for it if congregant phones are in scope.

Build stack

Vite and TypeScript, no UI framework, through Phase 2. TypeScript settles the build-step question before it is asked. Vite is near-zero configuration and provides Web Worker imports, WASM loading, and code splitting — all three of which this app needs specifically. That is the floor, not over-engineering.

No UI framework yet: through Phase 2 the product is one screen and roughly 1,500 lines, and a framework earns nothing against that. **Add Svelte at Phase 4**, when views multiply — setup, live, history, settings. Svelte rather than React because it compiles away, and the

byte budget is already committed to a ~1.5 MB corpus plus a ~40 MB model; also because the transcript repaints several times per second and a virtual DOM is pure overhead on that path. Deployment stays Vercel static and unchanged through Phase 3.

The one thing that breaks static-only hosting

Streaming server-side recognition needs a WebSocket, a long-lived process, and — on the GPU route — a GPU. Vercel functions provide none of these, so that path means a separate service on Fly.io, Railway, or a plain GPU VPS.

Which is itself an argument for the in-browser route: transformers.js keeps the product on pure static hosting through Phase 3. A server becomes unavoidable only when licensed translations arrive, since copyrighted text cannot be shipped to the client — a different reason, in a later phase.

Five phases

Ordered by dependency, not preference — each phase exists because the one after it is unsafe to start without it. Every phase ends at a gate you can measure rather than eyeball.

00

~1 week

De-risk the unknowns

Two questions can kill this product. Answer both before writing any product code.

- **Spike A — archaic English.** Web Speech API is trained on conversational speech; the KJV is *whithersoever* and *peradventure*. Record 20 readings, measure word error rate.
- **Spike B — Quranic recitation.** The real risk. The `ar-SA` model is trained on Modern Standard Arabic news and conversation, while recitation is Classical Arabic with tajweed: elongations, melodic delivery, specific articulation. Expect failure, and find out in week one rather than month four.
- **Build the eval harness now.** Audio in, predicted verse ID out, compared against ground truth. Everything downstream gets measured against it, so no change is ever a guess.
- **Free labelled data exists for both.** EveryAyah.com serves verse-by-verse recitation from many qaris — each file *is* one ayah with a known ID, roughly 6,236 labelled clips per reciter. LibriVox carries public-domain KJV audio of known text.

GATE A measured word error rate for both corpora, and a harness that scores a run automatically. If Spike B fails badly, Quran needs a fine-tuned model — a different budget and timeline, and better known now.

MEASURED · GATE MET

Harness built and run with `whisper-base`, the tier transformers.js would realistically load in a browser.

	SPIKE A — KJV ENGLISH	SPIKE B — RECITATION
Material	17 min continuous reading, 155 verses	120 ayah clips, 3 reciters
Word error rate	5.7%	60.0% median
Verse identification	95.2% in-book, 98.9% in order	88.3% acquire, 94.2% tracked

Spike A is closed. Archaic English was never the problem. **Spike B failed on transcription and passed on what matters** — recognition mangles recitation, but enough rare tokens survive for the ayah to resolve. Accuracy is flat across reciters (85%, 90%, 90%), so failures are ayah-specific, and length is the whole story:

GROUND-TRUTH LENGTH	N	ACQUIRE	TRACKED
1–4 words	21	61.9%	71.4%
5–9	30	80.0%	96.7%
10–19	45	100%	100%
20+	24	100%	100%

Every ayah of ten words or more was identified perfectly, by every reciter, at 60% word error rate. Every failure was eight words or shorter, and the worst are structural: 30:2 is two words and failed on all three reciters, because two words is not enough signal for any recogniser. In

production these never arrive alone — a reciter reads continuously, so context pins them.

The fine-tuned model branch is closed. Better recognition buys roughly four points; tracking buys nine and degrades far more gracefully.

01

~2–3 weeks

Make one corpus actually work

Full KJV, 31,102 verses. Still static, still no framework — the app is genuinely small at this stage.

- **Index.** Ship plain text (~4.5 MB, ~1.5 MB gzipped) and build the inverted index in a Web Worker at load. Shipping a prebuilt index buys about 200 ms and costs a format to maintain.
- **Scoring.** BM25, not raw overlap. It weights rare terms — precisely what the current scorer lacks. Pull candidates from the inverted index so you score roughly 150 verses instead of 31,102.
- **Tracking.** Position plus confidence. Locked, score only a window around the current verse; advance when the next verse outscores it; unlock when a global check beats the window by a margin for several consecutive frames. It is a small HMM — resist over-building it.
- **Fix the rolling transcript.** Use `e.resultIndex` and keep the last ~15 words. The present code rebuilds the entire session transcript on every event, which drives the match score below threshold after roughly 60–80 spoken content words. Tracking needs the fix anyway.
- **Put ASR behind an interface** so Phase 3 is a swap, not a rewrite.

GATE Harness reports precision above 95% on high-confidence matches across LibriVox samples. A number, not a feeling.

MEASURED · GATE MET

CONFIDENCE THRESHOLD	PRECISION	FRAMES RETAINED
≥ 0.00	95.2%	100%
≥ 0.25	98.1%	83.3%
≥ 0.50	99.2%	68.3%

Precision rises monotonically with confidence, so what the interface shows is discriminative rather than decorative. Confidence is the margin over the runner-up rather than an absolute score — what matters is whether anything else could plausibly be the answer.

What makes the number trustworthy. The harness measures Python; the browser runs TypeScript. A test pins them together across all 37,338 verses: a digest over every normalized verse, exact search rankings with scores, and a replay of the Spike A transcript through the shipping tracker. That replay reproduces the Python measurement exactly — 95.2% in-book, 98.9% in reading order.

The cross-check earned itself immediately, catching a bug that would have degraded Quran matching silently: the app derived matching text by normalizing display text, but the Quran's display text is Uthmani while the harness matches the simple edition, and the two diverge orthographically. Folding them still left 2,331 ayat apart. The plan already had the answer — match on simple, display Uthmani.

Measured in a browser. 31,102 verses indexed in 778 ms. Both corpora render, Arabic right-to-left with full diacritics. A spoken Psalm 23 acquires at 23:1 then follows 23:2 through 23:5 in tracked mode, confidence climbing 0.51 to 0.86. The core bundle is 5.3 kB gzipped, since transformers.js loads lazily and browsers taking the fast lane never fetch it.

Recognition since verified against real audio. The offline engine could not load its model at all on first contact with a real browser — an ONNX Runtime regression, fixed by pinning transformers.js to its 3.x line. With that done, English transcribes a spoken Psalm 23 as “The Lord is my shepherd. I shall not want. He may cathme to lie down in green pastures” — *maketh* mangled, exactly the error class the matcher exists to absorb — and resolves to Psalms 23:1 with 23:2 second. Arabic

transcribes a recitation of 1:1 as بسم الله الرحمن الرحيم, exactly its ground truth.

Since verified end to end on a real device. Microphone capture, the AudioWorklet, model load and recognition all run. Getting there surfaced one more defect: switching English to the English-only model left `language` and `task` in the generation options, which those models reject outright, so every pass failed the moment audio started flowing.

02

~1 week

Abstract the corpus

Both scriptures are ordered collections of numbered units, so they share one engine. They differ in ways that will cost you if collapsed.

	BIBLE	QURAN
Canonical text	the translation, for English readers	the Arabic; translation is secondary
Direction	left to right	right to left
Mode	reading	recitation

- Shared flat verse index, with a per-corpus reference formatter, normalizer, and renderer.
- **One engine, two adapters.** Do not build two apps; do not pretend they are the same thing either.

GATE The KJV path still passes every Phase 1 measurement after the refactor.

MEASURED · GATE MET

Each corpus now has one adapter holding its label, writing direction, recognition language, normalizer, and reference format. Those five facts had been flattened into ternaries spread across three files. Adding a third corpus is now adding an adapter, and the corpus picker builds itself from the adapter list rather than duplicating it in markup. Dependencies flow one way with no cycles.

The KJV path reports exactly the numbers it did before the refactor — 95.2% in-book, 98.9% in reading order, 98.1% precision above a 0.25 confidence threshold — and both corpora still agree with the Python harness across all 37,338 verses.

03

~3–4 weeks

Quran

Scope here depends entirely on what Phase 0 measured. Everything else is ready for it.

- **Text.** Tanzil.net, verified Uthmani. It ships both a simple variant — diacritics stripped, alif, ya and ta-marbuta normalized — and the full Uthmani. Match on simple, display Uthmani, write no normalization code of your own.
- **Rendering.** Right to left, a proper Arabic face with full harakat, optional transliteration and paired translation.
- **Recognition.** Ranked by what Phase 0 found: Web Speech API (free, weak on recitation) → whisper.cpp in WASM (offline, ~75 MB model, better but still soft on tajweed) → server-side Whisper (strongest general model, real cost) → a model fine-tuned on open recitation datasets (best here, real training effort).

GATE Correct ayah identified on held-out EveryAyah clips across at least three reciters — different voices, not three takes of one.

MEASURED · GATE MET

120 clips from a sample offset half a stride from the development set, so it shares no ayah with it, across Alafasy, Husary and Abdul Basit. Measured through the shipping TypeScript, replaying the exact transcripts the Python harness scored.

GROUND-TRUTH LENGTH	N	ACQUIRE	TRACKED
1–4 words	21	57.1%	85.7%
5–9	36	86.1%	91.7%
10–19	48	93.8%	100%
20+	15	100%	100%
all	120	85.8%	95.0%

Word error rate was 50% median. Accuracy is flat across reciters, and it generalizes: tracked accuracy on held-out clips is 95.0% against 94.2% on the development set. Everything from ten words up is identified perfectly once tracking has a position, and failures stay concentrated in short ayat.

Also shipped: Pickthall's 1930 translation, public domain since 2006, shown below the Arabic with attribution.

One bug worth recording. Extracting the normalizers into their own module let bidi reordering transpose two characters in the Arabic strip class, widening it to cover the whole alphabet. The normalizer returned an empty string for every input, and the class looked correct on screen. The whole suite still passed, because the Quran's matching text ships as its own file and the corpus digest never exercises that normalizer. Query normalization is now checked against the Python fixtures, and every Arabic codepoint in the module is an escape rather than a literal.

04

ongoing

Product shell

Accounts, saved sessions, projection output, offline mode.

- This is where a backend and a framework finally earn their place — **not before**. Through Phase 3 the app runs to roughly 1,500 lines and vanilla is the correct answer.
- Licensed translations force a server regardless, since you cannot ship copyrighted text to the client.

GATE A full service or khutbah followed end to end without losing lock or needing a hand on the keyboard.

MEASURED · GATE MET

The open question is answered: the operator holds the screen. The projector is a second document dragged to the second display, driven over `BroadcastChannel` with a heartbeat both ways. It holds no state, runs no recognition, and decides nothing — whatever the control window sends is what the room sees, so a fault in matching cannot reach the wall on its own.

What may reach the room unattended is the safety rule of this phase. Below the confidence bar, hold. An ambiguous reference holds however well it scored. An operator's own choice always goes.

Tracker excursions outside the passage	2, worst 8 consecutive frames
Longest unbroken run	177 frames
Frames where the wall showed a wrong verse	1 of 181 — 99.4% correct

The control view is allowed to wander; the screen is not. The gate measures the screen rather than the tracker, because that is what an audience sees.

No accounts and no backend, and no Svelte. This plan expected all three here. One machine running two windows in one browser has nothing to sync, and two views — one of them a hundred lines of display — do not earn a framework. Both follow from the audience rather than from deferral.

Two things that will bite

Licensing is a gating dependency, not a detail

The KJV is public domain in the US; UK Crown copyright is perpetual but not practically enforced abroad. Quran Arabic via Tanzil is clean. But the NIV, ESV and NLT are heavily copyrighted, cost real money, and cannot be shipped to the client — which forces a server the moment you add one. Quran translations vary by translator. **Design for pluggable translations from day one and launch on public domain only**, so the question is already answered when someone asks for the NIV.

A wrong verse is a trust failure, not a bug

The wrong verse on screen during a service is embarrassing. Wrong Quran text is offensive and can end the product outright. So: **precision over recall, always.** Never render a low-confidence match as though it were certain — show the reference, show alternatives, keep manual override one tap away. The existing confidence bar is the right instinct; make it authoritative rather than decorative.

Where you are actually differentiated

Church projection belongs to ProPresenter and EasyWorship. Quran recitation recognition belongs to Tarteel. The wedge is the thing neither does well: **real-time, hands-free, no operator.** Nobody clicking ahead trying to guess where the preacher is going next. That is genuinely underserved — build toward it rather than toward a general scripture viewer.

Decided, and what is left

Who holds the screen?

Phases 0 through 2 are identical either way, so this is not blocking — but it reshapes Phase 4 substantially, and it is the next thing worth settling. Same engine, three different products.

`congregant following along` `operator running projection`

`student drilling hifz`